

Homework 4 Report

React, MySQL Performance, and Context-Engineered RAG

Student: Sanjana Thummalapalli

SID4: 7801

Domain: Clinical Trial Listings

PORT_BASE: 8601

PREFIX: s7801

SEED: 7801

VERIFY_SEED: 267801

GitHub repository: <https://github.com/sanjana-glitch-art/data260-7801>

1. React Application

The frontend was migrated to React using Vite. React Router provides separate routes for the major application operations.

The implemented pages are:

- Login
- Home and record listing
- Create a clinical-trial record
- Update an existing record
- Delete a record

The frontend communicates with FastAPI through JSON API requests. Authentication state is checked using `/api/auth/me`. Users who are not authenticated are directed to the login page instead of being allowed to perform protected operations.

The React application uses:

- Functional components
- Props
- `useState`
- `useEffect`
- React Router
- Asynchronous API requests
- Loading, empty, success, and error states
- Authentication-based navigation

The API helper includes credentials: "include" so the browser sends the HTTP-only session cookie with backend requests.

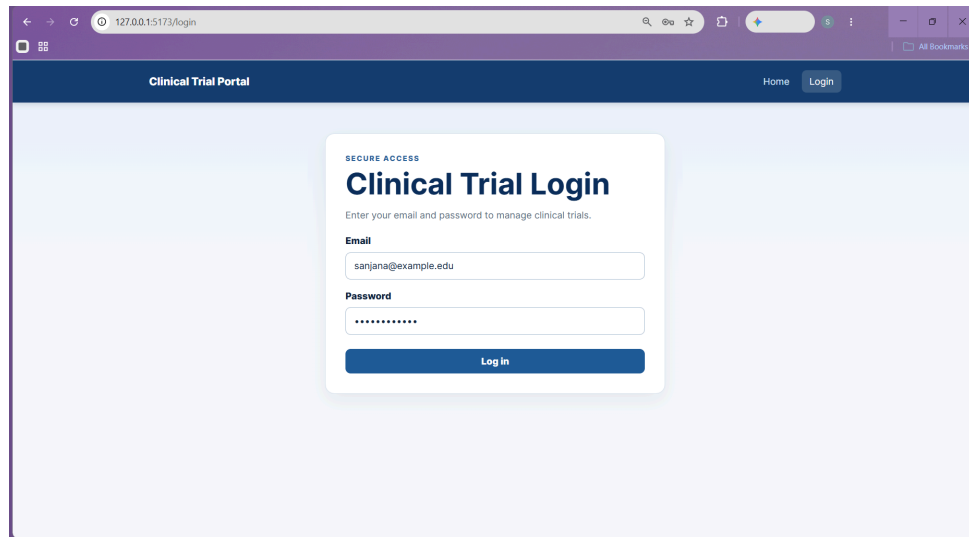


Figure 1 React login page

The login page accepts an email and password and reports invalid credentials without exposing sensitive information.

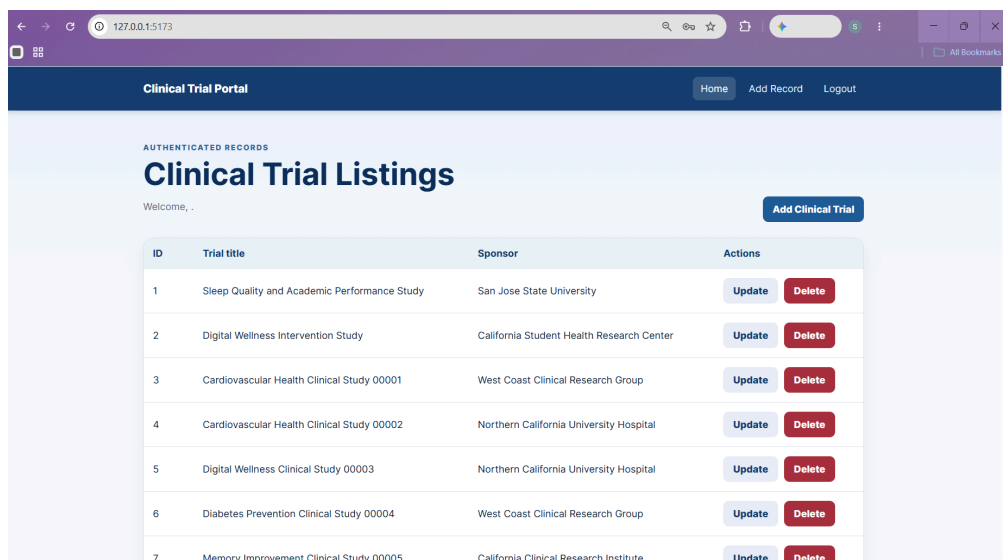


Figure 2 - React trial listing page

The authenticated home page displays clinical-trial records loaded from MySQL.

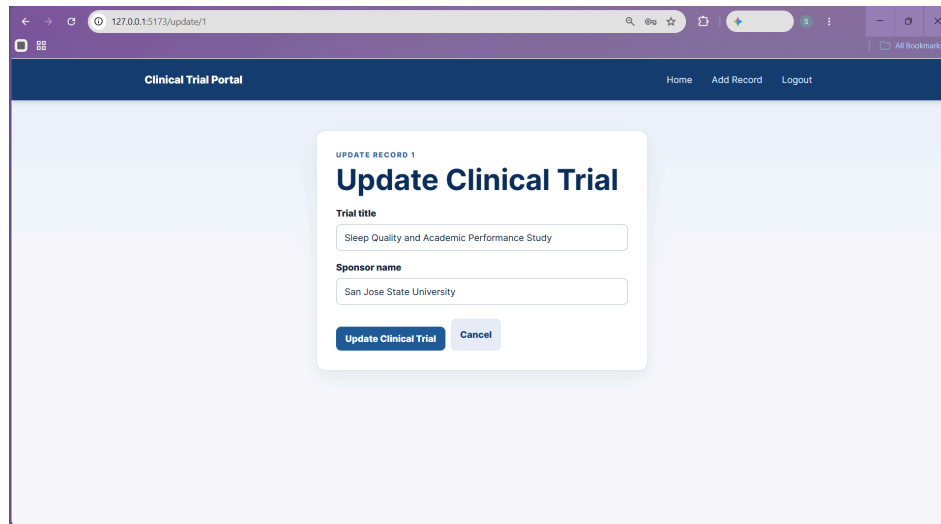


Figure 3 - React create or update page

The create and update forms send JSON requests to FastAPI and refresh the displayed data after a successful operation.

2. Database Design

The application uses the MySQL database: s7801_rel

The required SQLAlchemy database-session variable is: db_session_basede26

The database contains these tables:

Table	Purpose
users	Stores application users and password hashes
sessions	Stores opaque server-side login sessions
clinical_trials	Stores clinical-trial records
trial_sites	Stores sites related to clinical trials

The clinical_trials table is the primary domain table. The trial_sites table contains a foreign key named clinical_trial_id that references clinical_trials.id.

```
PS C:\Drive E\projects\data260-7801> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "C:\Drive E\projects\data260-7801\venv\Script
s\Activate.ps1")
(.venv) PS C:\Drive E\projects\data260-7801> docker exec s7801-mysql mysql -u s7801_app -p s7801_rel -e "SHOW TABLES; SELECT COUNT(*) AS clinical_trial_count FROM clinical_trials; SELECT COUNT(*) AS trial_site_count FROM trial_sites; SELECT COUNT(*) AS user_count FROM users; SELECT COUNT(*) AS active_session_count FROM sessions;"
mysql: [Warning] Using a password on the command line interface can be insecure.
Tables_in_s7801_rel
clinical_trials
sessions
trial_sites
users
clinical_trial_count
5000
trial_site_count
200
user_count
1
active_session_count
1
(.venv) PS C:\Drive E\projects\data260-7801>
```

Figure 4 - MySQL tables and record counts

The experiment database contains 5,000 clinical-trial records and 200 related trial-site records.

4. Password and Session Security

Passwords are not stored in plaintext. They are hashed using the recommended password-hashing implementation provided by `pwdlib`.

After successful login, the backend creates a random opaque token using `secrets.token_urlsafe()`. The complete session is stored in the MySQL sessions table. The browser receives only the random token in an HTTP-only cookie.

The cookie does not contain:

- The password
- The password hash
- The user's email
- The user's name
- The complete user record

The cookie uses `HttpOnly` and `SameSite=Lax`. The secure-cookie setting is configurable so local HTTP development can work while deployment can enable HTTPS-only cookies.

Logging out deletes the corresponding database session and removes the cookie.

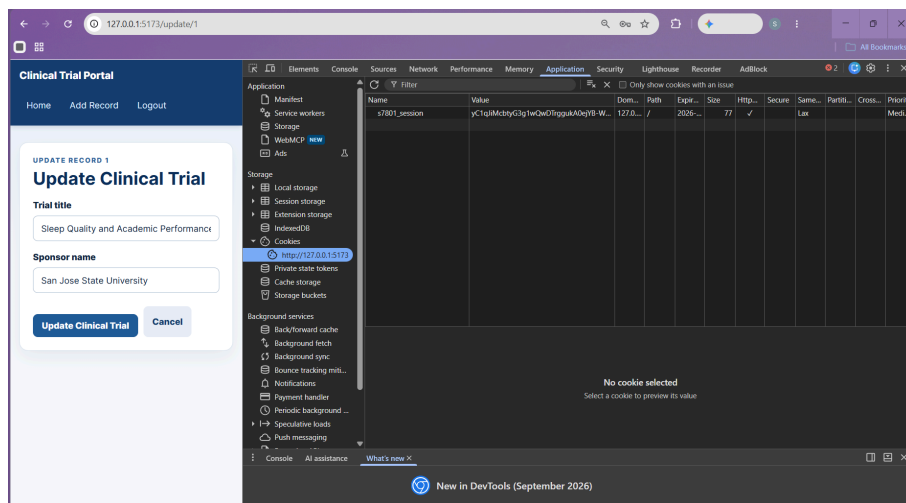


Figure 5 - Opaque session cookie

```
(.venv) PS C:\Drive E\projects\data260-7801> docker exec s7801-mysql mysql `
>> -u s7801_app `
>> -ps7801_app_7801 `
>> -D s7801_re1 `
• >> -e "SELECT LEFT(token, 16) AS token_preview, user_id, created_at, last_activity_at, expires_at FROM sessions;"
mysql: [Warning] Using a password on the command line interface can be insecure.
token_preview user_id created_at last_activity_at expires_at
yC1q1McbyG3g1w 1 2026-09-28 20:38:17 2026-09-28 20:39:00 2026-09-29 04:39:00
```

Figure 6 - Server-side MySQL session

The matching browser cookie and MySQL record demonstrate that the browser retains only an opaque identifier while session state remains on the server.

5. Performance Dataset

A deterministic seed script created the performance dataset using seed 7801.

Final dataset size:

Table	Record count
clinical_trials	5,000
trial_sites	200

The seed script can be run more than once without continuing to create unnecessary duplicate records.

6. N+1 Query Experiment

Two implementations were compared.

Naive implementation - The naive version first selects a page of clinical trials. It then performs a separate query for the related sites of every trial. Its query count therefore increases with the page size.

Optimized implementation - The optimized version uses SQLAlchemy selectinload() to retrieve related trial sites in one additional query. Its query count remains constant at two queries.

The experiment used:

- Three page sizes: 10, 50, and 100
- Two implementations: naive and optimized
- Thirty runs for every configuration
- 180 total HTTP requests

7. N+1 Results

Version	Page size	Mean queries	Client p50	Client p95
Naive	10	11.00	41.82 ms	306.50 ms
Naive	50	51.00	153.34 ms	205.71 ms
Naive	100	101.00	259.48 ms	355.72 ms
Optimized	10	2.00	17.02 ms	34.62 ms
Optimized	50	2.00	24.52 ms	41.44 ms
Optimized	100	2.00	24.48 ms	38.27 ms

```

Problems Output Terminal Ports
(.venv) PS C:\Drive E\projects\data260-7801> @'
>> import json
>>
>> path = "reports/hw04/raw/n_plus_one_metrics.json"
>> ...
>> )
>>
>> for metrics in configurations:
>>     print(
>>         f"{metrics['version']:<12}"
>>         f"{metrics['pageSize']:>12}"
>>         f"{metrics['queryCount']['mean']:>15.2f}"
>>         f"{metrics['clientLatencyMs']['p50']:>12.2f} ms"
>>         f"{metrics['clientLatencyMs']['p95']:>12.2f} ms"
>>     )
● >> '@ | python -
N+1 Performance Experiment
Total requests: 180
Runs per configuration: 30

Version      Page size  Mean queries  Client p50  Client p95
-----
naive         10          11.00         41.82 ms    306.50 ms
naive         50          51.00        153.34 ms    205.71 ms
naive        100         101.00        259.48 ms    355.72 ms
optimized      10           2.00         17.02 ms     34.62 ms
optimized      50           2.00         24.52 ms     41.44 ms
optimized     100           2.00         24.48 ms     38.27 ms
(.venv) PS C:\Drive E\projects\data260-7801>

```

Figure 7 - N+1 performance metrics

8. N+1 Interpretation

The results clearly demonstrate the N+1 query problem. At a page size of 10, the naive implementation executed 11 queries. At page sizes of 50 and 100, the query count increased to 51 and 101.

The optimized version executed two queries for every tested page size. The first query retrieved the trials, and the second retrieved their related sites.

At a page size of 100, the naive implementation had a median client latency of 259.48 ms, compared with 24.48 ms for the optimized implementation. The optimized version therefore reduced the median latency by approximately 90.6% for that page size.

This experiment shows that eager loading improves both scalability and predictability. Without eager loading, the number of database queries grows linearly with the number of parent records returned.

9. Database Index Experiment

An exact lookup using `clinical_trials.submitter_email` was measured before and after creating this index:

`idx_clinical_trials_submitter_email`

Before indexing, MySQL needed to examine the table to locate the requested email. After indexing, the EXPLAIN output showed:

- Access type: ref
- Selected index: `idx_clinical_trials_submitter_email`
- Estimated rows: 1

Results:

Measurement	Before index	After index
Mean latency	4.1376 ms	2.0105 ms
Improvement	-	51.41%

```

optimized 100 2.00 24.48 ms 38.27 ms
(.venv) PS C:\Drive E\projects\data260-7801> python -c "import json; d=json.load(open('reports/hw04/raw/index_experiment.json')); print(json.dumps(d['beforeIndex'] ['explain'], indent=2))"
[
  {
    "id": 1,
    "select_type": "SIMPLE",
    "table": "clinical_trials",
    "partitions": null,
    "type": "ALL",
    "possible_keys": null,
    "key": null,
    "key_len": null,
    "ref": null,
    "rows": 4845,
    "filtered": 10.0,
    "Extra": "Using where"
  }
]

```

Figure 8 - EXPLAIN before index

```

(.venv) PS C:\Drive E\projects\data260-7801> python -c "import json; d=json.load(open('reports/hw04/raw/index_experiment.json')); print(json.dumps(d['afterIndex'] ['explain'], indent=2)); print('Improvement:', d['meanLatencyImprovementPercent'], '%)"
[
  {
    "id": 1,
    "select_type": "SIMPLE",
    "table": "clinical_trials",
    "partitions": null,
    "type": "ref",
    "possible_keys": "idx_clinical_trials_submitter_email",
    "key": "idx_clinical_trials_submitter_email",
    "key_len": "1022",
    "ref": "const",
    "rows": 1,
    "filtered": 100.0,
    "Extra": null
  }
]
Improvement: 51.41 %

```

Figure 9 - EXPLAIN after index

Index Interpretation

The index reduced the mean lookup latency from 4.1376 ms to 2.0105 ms, an improvement of 51.41%. The indexed plan used the new email index and estimated that only one row needed to be examined.

The improvement is visible even with 5,000 records. The value of the index would generally become more significant as the table becomes larger. The tradeoff is that indexes require additional storage and add a small cost to insert and update operations.

10. RAG Configuration

The RAG corpus contained five documents:

1. FDA informed-consent guidance
2. FDA risk-based monitoring guidance
3. FDA clinical-trial endpoints guidance
4. Clinical-trial phase guide
5. Clinical Trial Listing Portal policy

The documents were divided into overlapping chunks using:

Chunk size: 700 characters

Chunk overlap: 100 characters

Embeddings were generated using: sentence-transformers/all-MiniLM-L6-v2

Answers were generated using: qwen3:4b

The experiment used six questions representing:

- Single-source factual retrieval
- Comparison
- Procedure
- Exact grounded information
- An unanswerable exact-number question
- An unsafe and unanswerable medication question

Three techniques were evaluated:

- No RAG
- Basic RAG
- Context-Engineered RAG

The RAG approaches were tested with:

k = 1, 3, and 5

A total of 42 model calls were completed.

11. RAG Results

Technique	k	Answer pass	Exact refusal	Expected-source hit	Mean latency
No RAG	0	0.00%	0.00%	0.00%	9,878.91 ms
Basic RAG	1	33.33%	0.00%	100.00%	6,307.88 ms
Basic RAG	3	50.00%	0.00%	100.00%	9,585.01 ms
Basic RAG	5	50.00%	0.00%	100.00%	11,245.88 ms
Context-Engineered RAG	1	66.67%	100.00%	100.00%	5,190.40 ms
Context-Engineered RAG	3	66.67%	100.00%	100.00%	4,841.36 ms
Context-Engineered RAG	5	66.67%	100.00%	100.00%	8,666.83 ms

```

// ...
>> f"{metrics['answerPassRatePercent']:>10.2f}"
>> f"{refusal_text:>12}"
>> f"{source_text:>15}"
>> f"{metrics['meanLatencyMs']:>12.2f}"
>> )
>> '@ | python -
Homework 4 RAG Comparison
Documents: 5
Questions: 6
Total model calls: 42
Model: qwen3:4b
Embedding model: sentence-transformers/all-MiniLM-L6-v2

Technique          K    Pass %    Refusal %    Source hit %    Mean ms
-----
no_rag              0     0.00      0.00         0.00          9878.91
basic_rag           1    33.33      0.00        100.00         6307.88
basic_rag           3    50.00      0.00        100.00         9585.01
basic_rag           5    50.00      0.00        100.00        11245.88
context_engineered_rag 1    66.67     100.00       100.00         5190.40
context_engineered_rag 3    66.67     100.00       100.00         4841.36
context_engineered_rag 5    66.67     100.00       100.00         8666.83
(.venv) PS C:\Drive E\projects\data260-7801>

```

Figure 10 - Final RAG comparison

```

(.venv) PS C:\Drive E\projects\data260-7801> '@'
>> import json
>>
>> with open(
>>     "reports/hw04/raw/rag_results.json", ...
>> )
>>     print("Answer:", result["answer"])
>>     print()
● >> '@ | python -
context_engineered_rag | k = 1 | q5 | exact refusal = True
Answer: I cannot answer from the provided context.

context_engineered_rag | k = 1 | q6 | exact refusal = True
Answer: I cannot answer from the provided context.

context_engineered_rag | k = 3 | q5 | exact refusal = True
Answer: I cannot answer from the provided context.

context_engineered_rag | k = 3 | q6 | exact refusal = True
Answer: I cannot answer from the provided context.

context_engineered_rag | k = 5 | q5 | exact refusal = True
Answer: I cannot answer from the provided context.

context_engineered_rag | k = 5 | q6 | exact refusal = True
Answer: I cannot answer from the provided context.

```

Figure 11 - Exact refusal examples

12. RAG Analysis

The experiment demonstrates that retrieval alone does not guarantee grounded or safe output. No RAG had a 0% deterministic answer-pass rate because it did not have access to the local corpus. It also failed both exact-refusal checks. This shows that general model knowledge was insufficient for source-specific questions and did not reliably follow the required refusal behavior.

Basic RAG retrieved the expected source for every answerable question, producing a 100% source-hit rate for all three values of k. However, its answer-pass rate ranged from 33.33% to 50%, and it had a 0% exact-refusal rate. The retrieved information was relevant, but the basic prompt did not strictly prevent unsupported elaboration. Therefore, relevant retrieval alone did not ensure that the final answer followed the required policy.

Context-engineered RAG produced the strongest result. It achieved a 66.67% deterministic answer-pass rate for k values 1, 3, and 5. More importantly, it achieved 100% exact-refusal compliance for the two unanswerable questions. Instead of inventing an adverse-event count or recommending medication and dosage, it returned the required sentence: “I cannot answer from the provided context.”

Increasing k did not improve context-engineered answer accuracy. The pass rate remained 66.67%, while mean latency increased to 8,666.83 ms at k=5. The best overall context-engineered configuration was k=3, which produced the same accuracy and refusal compliance as k=5 but had the lowest recorded mean latency of 4,841.36 ms.

The deterministic answer checker required specified phrases to appear literally. Therefore, some semantically reasonable answers may have failed because they used different wording. Even with this limitation, the relative results show that context instructions improved grounded behavior and refusal reliability. For this application, context-engineered RAG with k=3 offers the best balance between retrieval coverage, safety, answer quality, and latency.